

1.Introduction

In this project, we analyze a dataset of trending YouTube videos to gain insights into what makes a video popular in terms of views, likes, and comments. The dataset contains various features about the videos, such as title, category, number of views, likes, dislikes, and comment count, along with the publish time. This topic was chosen due to the massive impact of YouTube on modern media consumption and the importance of understanding engagement trends for content creators and marketers.

Dataset Source: [Insert dataset link here]

Number of records: [Insert number of rows after loading the dataset]

Dataset Columns:

video_id: Unique identifier for the video.

title: Title of the YouTube video.

category: Categorical ID representing the genre/type of video.

views: Number of views.

likes: Number of likes.

dislikes: Number of dislikes.

comment_count: Number of comments.

publish_time: Date and time the video was published.

tags: List of video tags.

Field of Interest: The main focus is to explore how different factors (category, publish time,etc.) relate to the popularity of the videos.

2.Loading Libraries and Importing Data

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Load dataset
```

```
df=pd.read_csv('your_dataset.csv')
```

```
# Display first rows
```

```
print(df.head())
```

```
# Describe the dataset
```

```
print(df.describe())
```

Explanation of describe() output: The describe() function provides statistical summaries of the numerical columns including:

count: Number of non-null values.

mean: Average value.

std: Standard deviation.

min and max: Minimum and maximum values.

25%, 50% (median), 75%: Percentile values for distribution.

3.Data Cleaning

1.Drop unnecessary columns:

Dropped video_id, tags as they don't contribute to our analysis.

```
df.drop(['video_id','tags'],axis=1,inplace=True)
```

2. Rename columns:

Renamed comment_count to comments for simplicity.

```
df.rename(columns={'comment_count': 'comments'}, inplace=True)
```

3. Remove duplicates:

```
df.drop_duplicates(inplace=True)
```

4.Handle missing data:

```
df.dropna(inplace=True)
```

5.Create new columns:

Extract year from publish_time.

```
df['publish_time']=pd.to_datetime(df['publish_time'])
df['published_year'] = df['publish_time'].dt.year
```

4.Pandas Aggregate Functions

Meaningful usage of aggregate functions:

Mean

```
mean_likes = df['likes'].mean()
```

Median

```
median_comments = df['comments'].median()
```

Standard Deviation

```
std_views = df['views'].std()
```

Variance

```
varlikes = df['likes'].var()
```

These statistics help us understand the central tendency and variability of video engagement metrics.

5.Data Visualization

1. Bar Chart-Number of Videos per Category

```
category_counts = df['category'].value_counts()
category_counts.plot(kind='bar')
plt.title('Number of Videos per Category')
plt.xlabel('Category')
plt.ylabel('Count')
plt.xticks(rotation=45)

plt.grid(axis='y', linestyle='--')
plt.show()
```

2. Bar Chart-Top 10 Videos by Views

```
top_views = df.sort_values('views', ascending=False).head(10)
plt.bar(top_views['title'], top_views['views'])
plt.title('Top 10 Videos by Views')
plt.xticks(rotation=90)
plt.show()
```

3. Pie Chart-Top 5 Categories

```
top_categories = df['category'].value_counts().head(5)
plt.pie(top_categories, labels=top_categories.index, autopct='%1.1f%%')
plt.title('Top 5 Categories')
plt.axis('equal')
plt.show()
```

4. Histogram-Views Distribution

```
plt.hist(df['views'], bins=30)
plt.title('Distribution of Video Views')
plt.xlabel('Views')
plt.ylabel('Frequency')
plt.show()
```

Each graph includes a title, labels, formatting, and doesn't repeat insight areas

5-scatter plot diagram .

Conclusion

This analysis provided valuable insights into YouTube video trends. The visualization and aggregation showed how different categories perform and how engagement metrics like likes and views vary. The project strengthened our understanding of data preprocessing, statistical analysis, and effective visualization.